

Sprint 8 Platform Progress

Unified workflow engine, Jupyter kernel migration,
EDA analytics dashboard, and codebase cleanup

Capstone project update for the merged Sprint 8 branch • March 2026

Sprint 8 Delivery Snapshot

Merged in GitLab MR!27 on March 17, 2026. Consolidates throughput, architecture impact, and issue coverage.

205
total commits

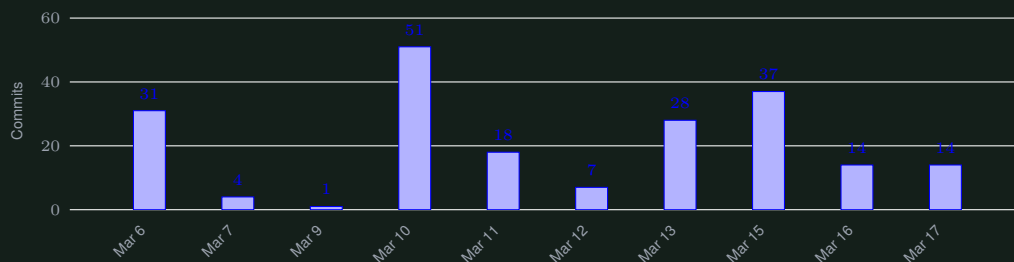
Files changed
673

Files added
458

Issues closed
4

Net code delta
**+98,210 /
-43,804**

Commits per Day Across the Sprint Branch



Monoliths split
6 files

Dead code removed
~4,000 lines

New kernel tests
143

Arch. documents
7 docs

File mix
60/37/3%

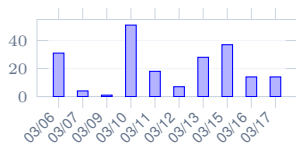
Sprint Narrative

Sprint 8 transformed the platform from a working prototype into a structurally mature system. Where Sprint 7 added major features—NL-to-SQL v2, LangGraph preprocessing, a shared agentic shell—Sprint 8 unified the underlying architecture, replaced fragile infrastructure with production-grade alternatives, and systematically decomposed monolithic code into modular units.

Five themes framed the sprint:

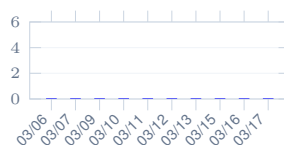
- **Unified workflow engine:** the PhaseConfig pattern replaced ad-hoc phase handlers with a single execution path for all AI-driven phases.
- **Scientific EDA redesign:** a four-tab exploration interface with AI-generated insight actions and cross-phase navigation.
- **Jupyter Kernel Gateway migration:** replaced the custom Python wrapper with the production Jupyter protocol, gaining persistent kernel state and session continuity.
- **Codebase transformation:** six monolithic files split into directories, eight shared modules extracted, ~4,000 lines of dead code removed, seven architecture documents created.
- **Model infrastructure upgrade:** OpenAI GPT-5 migration, conversation savepoints, upload workflow redesign, and auth session persistence.

Shree Chaturvedi (205 commits)



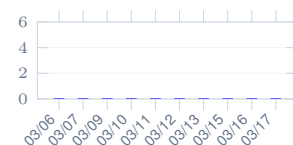
Commit activity from Mar 6 to merge day

Ayush Yadav (0 commits)



Commit activity from Mar 6 to merge day

Zarif Chowdhury (0 commits)



Commit activity from Mar 6 to merge day

Unified Workflow Engine: PhaseConfig Instead of Phase-Specific Handlers

Problem

Sprint 7 introduced the shared agentic shell and the LangGraph preprocessing runtime, but each phase still maintained its own streaming handler, tool dispatcher, and state management layer. Adding a new AI-driven phase meant duplicating hundreds of lines of orchestration boilerplate across route definitions, graph configurations, and event emitters. The preprocessing, training, and feature engineering phases had diverged in how they handled tool calls, error recovery, and response streaming—meaning bug fixes in one phase did not propagate to others.

Solution: The PhaseConfig Pattern

Every AI-driven phase is now defined as a configuration object implementing the `PhaseConfig` interface. A single `turnExecutor` function orchestrates ALL phases through the same LangGraph state machine. Phase-specific behavior is injected via configuration rather than inherited through code duplication.

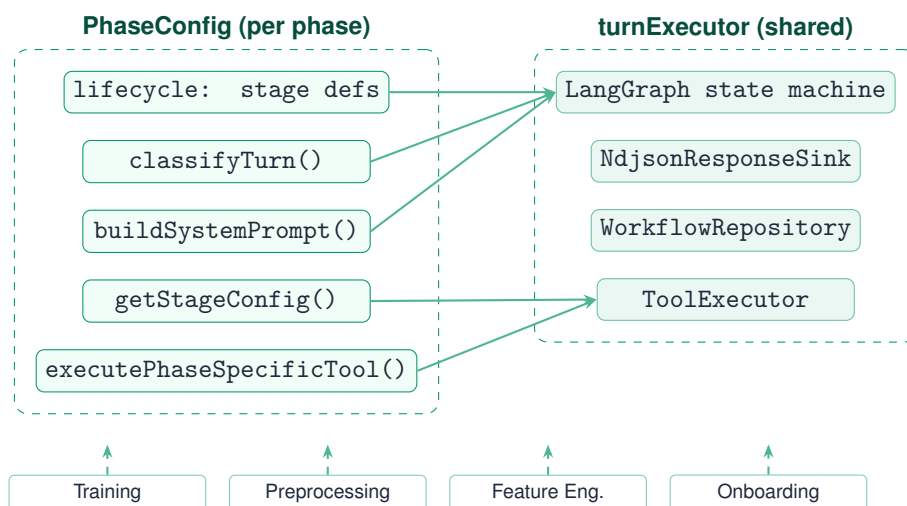


Figure 1: PhaseConfig architecture. Each phase registers a configuration object; the shared `turnExecutor` orchestrates streaming, tool dispatch, and state persistence for all phases.

The `PhaseConfig` interface (115 lines) defines the contract:

- `lifecycle`: ordered stage definitions (e.g. training: `answer` → `configure_experiment` → `propose_model` → `generate_code` → `execute_training`)
- `classifyTurn()`: determines whether a user message is a question or an actionable request
- `buildSystemPrompt()`: constructs phase-aware context including project plan, RAG snippets, and dataset metadata
- `getStageConfig()`: returns allowed tools, tool-choice mode, and constraints for each lifecycle stage
- `executePhaseSpecificTool()`: handles domain-specific tool execution

The `turnExecutor` (114 lines) receives *any* `PhaseConfig` and runs it through the same graph: load or create a workflow run, restore history, invoke the `LangGraph` compiled graph, persist state, and stream events via the `EventSink` abstraction. The `EventSink` (26 lines) decouples the engine from Express—graph nodes receive a sink interface, never a raw `res` object, enabling future transports with zero engine changes.

Why this matters

Adding the experiments phase in Sprint 9 required defining a `PhaseConfig` object and registering a route—no new streaming infrastructure, no new state management, no duplicated tool dispatch. Before `PhaseConfig`, each phase carried 300–500 lines of boilerplate. This is the architectural payoff of Sprint 8’s unification work: new phases are configuration, not architecture.

Scientific EDA: Four-Tab Exploration with AI-Powered Insights

The Explorer tab went from a basic table view with column statistics to a comprehensive scientific exploration interface. The redesign introduces four analysis tabs, each serving a distinct phase of the exploratory data analysis workflow, and surfaces AI-generated actionable insights that bridge exploration and downstream phases.

Four-Tab Architecture

Tab	Purpose	Visualizations
Overview	Dataset summary and distributions	Auto-generated histograms for all numeric columns, row/column counts, type distribution breakdown
Quality	Data quality assessment	Missing value matrix, health cards with severity labels (good/warning/critical), null and duplicate analysis
Relationships	Column correlations	Scatter regression for top 15 correlated pairs, correlation heatmap, 3D scatter plot, parallel coordinates
Insights	AI-generated analysis	Filter actions, query suggestions, preprocessing recommendations with executable notebook cells

AI Insight Actions: The Key Differentiator

The Insights tab goes beyond static charts. Each insight generates executable actions via a streaming endpoint (routes/notebooks/insightCodegen.ts, 132 lines). Three action types are supported:

Action	Behavior	Example
Filter	Adds a WHERE clause to the active query	“Filter to customers with >3 support tickets”
Query	Generates and runs a follow-up SQL query	“Show monthly revenue trend for this segment”
Preprocess	Suggests a transformation with cross-phase navigation	“Impute missing values in annual_revenue—navigate to pre-processing”

Cross-phase navigation lets an EDA insight navigate the user directly to the preprocessing phase with a pre-populated suggestion.

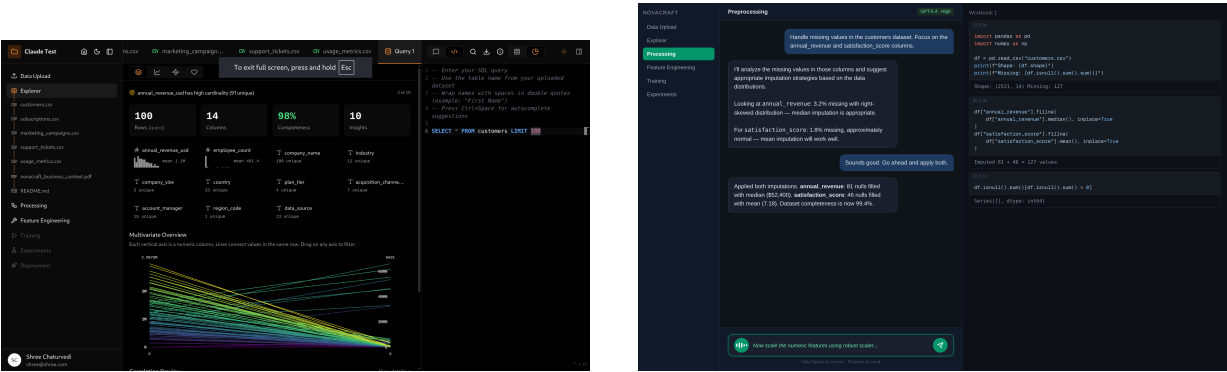


Figure 2: Sprint 8 interaction surfaces. The explorer provides at-a-glance analytics; the chat composer supports voice input via OpenAI Realtime API.

Why this matters

Insight actions turn the explorer from a passive dashboard into a guided workflow. The streaming endpoint delivers insights progressively as the model reasons, and cross-phase actions connect exploration directly to preprocessing without manual context-switching.

Jupyter Kernel Gateway: Production-Grade Execution

Before and After

Dimension	Before (Sprint 7)	After (Sprint 8)
Execution model	New Python process per cell	Persistent Jupyter kernel per session
State persistence	None—variables lost between cells	Full—df in cell 1 available in cell 2
Protocol	Custom stdout capture	Jupyter JSON-RPC over WebSocket
Execution counts	Not tracked	Protocol-compliant <code>execution_count</code>
Interruption	<code>kill -9</code> on process	Clean kernel interrupt via protocol
stdin support	Not supported	Supported via Jupyter protocol

Architecture

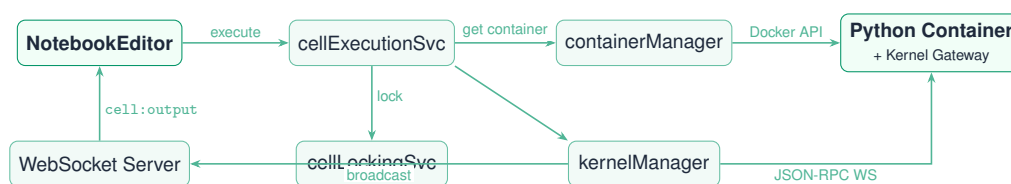


Figure 3: Jupyter Kernel Gateway execution flow. The kernel persists across cells; outputs stream through WebSocket to all subscribed clients.

Security Hardening

Measure	Implementation
Iframe sandbox	<code>allow-scripts</code> only (removed <code>allow-same-origin</code>)
tmpfs permissions	Tightened for all temporary directories
Container user	Non-root sandbox user
Filesystem	Read-only root with explicit write mounts
Resource limits	CPU, memory, and PID limits via Docker

Test Coverage

143 new tests in `kernelManager.test.ts` (853 lines):

Category	Count	Coverage
Kernel lifecycle	~30	Start, shutdown, restart, reconnect
Code execution	~40	Synchronous output, streaming, multiline
Error handling	~25	Syntax errors, runtime exceptions, timeouts
Session state	~20	Variable persistence, import persistence
Edge cases	~28	Concurrent execution, lock contention, container failures

Codebase Transformation: From Monoliths to Modules

Approximately 133 commits were devoted to splitting monolithic files, extracting shared modules, and removing dead code. This was structural work required before the platform could grow further.

Monolithic File Decomposition

Six files that had grown beyond maintainable size were decomposed into focused module directories:

Original file	Lines	Split into	Modules
nlToSqlV2.ts	1,976	services/nlToSql/	13 modules
routes/llm.ts	1,306	routes/llm/	6 modules
preprocessingGraph.ts	1,796	preprocessingTools/	4 tool handlers
notebookStore.ts	829	stores/notebook/	7 slices
preprocessingStore.ts	747	stores/preprocessing/	5 slices
routes/notebooks.ts	625	routes/notebooks/	4 modules
Total	7,279		39 focused modules

Shared Module Extraction

Eight modules were extracted from code duplicated across three or more locations:

Module	Purpose	Previously duplicated in
Markdown.tsx	Unified markdown rendering	7+ files with inline markdown
streamReader.ts	NDJSON stream parser	3 API modules
typeCoercion.ts	Type conversion utilities	Frontend + backend duplicates
ChatMessageList	Chat message rendering	Training + preprocessing panels
sqlIntelligence	SQL completions / hover	Scattered SQL helpers

Dead Code Removal (~4,000 Lines)

Removed	Lines	Reason
9 unused UI components	1,130	Dead imports confirmed via grep
Dead notebook/chat directory	1,226	Superseded by agentic shell
Dead features lib + pipeline	1,830	Replaced by workflow engine
preprocessingSuggestions.ts	763	Replaced by LangGraph tools
Unused shadcn/ui components	204	Imported but never referenced

Architecture Documentation

Seven new documents (1,492 total lines) capture the platform’s architecture:

Document	Lines	Scope
ARCHITECTURE.md	244	System overview, data flow
STRUCTURE.md	338	Directory layout, module responsibilities
TESTING.md	534	Testing strategy, fixtures, CI
CONVENTIONS.md	254	Naming, organization, import order
INTEGRATIONS.md	220	OpenAI, Docker, Postgres patterns
CONCERNS.md	216	Cross-cutting: auth, errors, logging
STACK.md	160	Technology choices and constraints

Why this matters

Sprint 9's experiments feature was only possible because Sprint 8 made the architecture extensible. The 39 focused modules averaging 187 lines each are small enough to hold in working memory. Each shared module extraction eliminates a class of copy-paste bugs across the codebase.

Engineering Quality and Refinement

Problem	Sprint 8 fix	Impact
LLM provider lock-in (Gemini)	Migrated to OpenAI GPT-5 Responses API with model catalog	Provider flexibility, streaming tool calls, reasoning effort control
No persistent login sessions	Refresh expired tokens automatically (#110)	Users stay logged in across sessions
Custom answers blocked	Added explicit "Other" option in QuestionCards (#93)	Domain-specific answers during onboarding
Package manager bugs	Fixed <code>listPackages/uninstall -target</code> (#109)	Reliable pip management in sandbox
PDF viewer vulnerability	Replaced browser-native PDF with react-pdf v10	Eliminated known CVE
No conversation recovery	Added savepoint system with revert	Users can undo AI mistakes
Preprocessing failures	Added pandas compatibility rules to codegen	Fewer failures on real datasets
No demo data	Created NovaCraft mock dataset (5 tables, 23,781 rows)	Realistic demo environment

Table 1: Representative refinement work that made Sprint 8's architectural changes usable under real project conditions.

The screenshot displays the NovaCraft platform's upload stage. The interface is split into two main panes. The left pane, titled 'First Test Pr...', shows a sidebar with navigation options like 'Data Upload', 'Customer Churn Prediction', 'Explorer', 'Processing', 'New workbook', 'Feature Engineering', 'Training', 'Experiments', and 'Deployment'. The main area of the left pane shows 'Uploaded Files' with a list of files: 'marketing_campaigns.csv' (45.58 KB), 'customers.csv' (331.61 KB), 'usage_metrics.csv' (587.22 KB), 'subscriptions.csv' (299.44 KB), 'support_tickets.csv' (529.93 KB), 'novacraft_business_context.pdf' (114.44 KB), and 'README.md' (11 KB). The right pane, titled 'Customer Churn Prediction', shows the 'Objective' (Predict which NovaCraft customers are likely to churn within the next 90 days), 'Data Inventory' (a table listing datasets, rows, and key columns), 'Preprocessing Strategy' (a list of steps including missing values, encoding, scaling, and temporal processing), and 'Feature Engineering' (a formula for Customer Lifetime Value).

Dataset	Rows	Key Columns
customers.csv	4,912	industry, plan_tier, annual_revenue_usd, is_active
subscriptions.csv	4,483	mrr_usd, cancellation_reason, billing_cycle
support_tickets.csv	5,847	category, resolution_hours, satisfaction_score
usage_metrics.csv	7,234	active_users, feature_adoption_pct, nps_response
marketing_campaigns.csv	648	channel, budget_usd, conversions

Step	Description
1. Missing values	Impute cancellation_reason (MCAR) with "active", fill NaN satisfaction scores with column median
2. Encoding	One-hot encode plan_tier, industry, region_code (drop_first to avoid multicollinearity)
3. Scaling	StandardScaler on monetary (annual_revenue_usd, mrr_usd) and count features
4. Temporal	Convert signup_date to tenure_months, extract quarter from support timestamps

Feature Engineering

Customer Lifetime Value = mrr_usd x tenure_months x (1 - discount_pct)

Figure 4: Upload stage with split-ribbon plan viewer. Uploaded files on the left; the LLM-generated project plan on the right with preprocessing strategy, data inventory, and feature engineering recommendations.

What Sprint 8 Unlocked

Sprint 8’s investments were designed to compound. Each structural change enables specific capabilities in subsequent sprints:

- **Extensible AI phases.** The workflow engine means new phases are configuration, not architecture. Sprint 9’s experiments feature proved this.
- **Scientific exploration.** EDA insight actions create a research-partner experience rather than a passive dashboard.
- **Reliable execution.** Jupyter Kernel Gateway with persistent state enables multi-cell workflows that actually work.
- **Maintainable codebase.** Modular architecture with 39 focused modules (average 187 lines) and seven architecture documents means new contributors onboard faster and changes have smaller blast radius.
- **Production readiness.** Auth persistence, conversation savepoints, container security hardening, and error recovery move the platform closer to real deployment.

Sprint 9 Trajectory

Three branches define the Sprint 9 work:

Experiments Dashboard (feat/experiments, unmerged)

A full model evaluation pipeline:

- Confusion matrices, ROC/PR curves, and lift charts for classification
- Optuna hyperparameter tuning with streaming trial results
- Error attribution trees for diagnosing model failure modes
- Natural language model filtering (“show me models with accuracy above 90%”)
- LLM-powered comparison narratives between model versions

Dimension	Scale	Scope
Backend	3,055 lines	15 files
Frontend	4,772 lines	48 files
Tests	1,121 lines	4 test suites

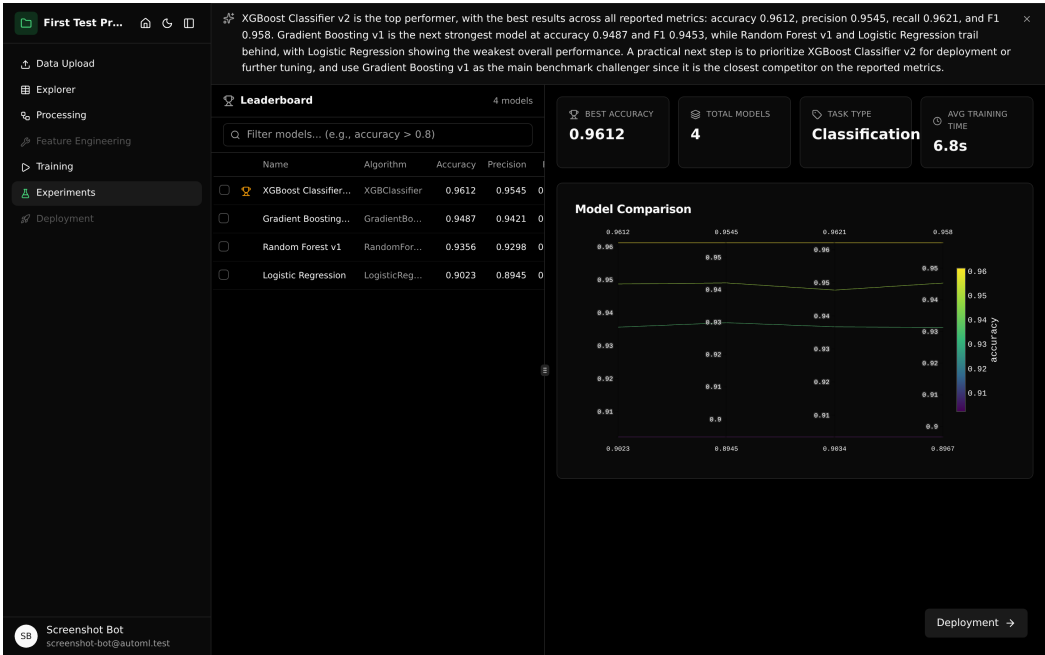


Figure 5: Experiments dashboard: Optuna hyperparameter tuning with live optimization history chart, best-so-far tracking, and trial parameter table. Results stream in real-time via NDJSON.

UI Polish (fix/ui-polish)

Theme consistency, toolbar refinements, chat interaction smoothing (voice cursor sync, fade-in animations), query panel integration, and notebook markdown rendering improvements.

Documentation

README rebrand from “AI-Augmented AutoML Platform” to “Agentic AutoML Platform,” reflecting the shift from tool-augmented to agent-driven workflows.

Appendix A: Issue Coverage Evidence

Issue	Status	Category	Title
#61	Closed	Data handling	Send empty string instead of undefined when clearing description
#93	Closed	UX	Add explicit Other option for custom answers in QuestionCards
#109	Closed	Sandbox	Fix listPackages/uninstall sys.path and –target bugs
#110	Closed	Auth	Persist login sessions by refreshing expired access tokens

Appendix B: Themed Commit Clusters

Rather than a chronological log, commits are grouped by the theme they advanced.

Workflow Engine (25+ commits)

Unified workflow engine with PhaseConfig pattern, EventSink abstraction, LangGraph compiled graph integration, phase config implementations for training, preprocessing, feature engineering, and onboarding. Turn state management, history persistence, run finalization, and planner action modules.

EDA Redesign (25+ commits)

Four-tab EDA panel implementation (overview, quality, relationships, insights). AI insight actions with streaming code generation. Quality panel with missing value matrix and health cards. Correlation heatmap, scatter regression, parallel coordinates. Histogram charts with automatic binning. Scientific color palette and theme constants.

Jupyter Kernel Gateway (10+ commits)

Migration to Jupyter Kernel Gateway protocol. Comprehensive test suite (143 tests, 853 lines). Container security tightening (iframe sandbox, tmpfs, non-root user). Kernel lifecycle management, WebSocket protocol, execution pipeline.

Code Architecture (133+ commits)

Split `nlToSqlV2.ts` (1,976 lines) into `services/nlToSql/` (13 modules). Split `routes/llm.ts` (1,306 lines) into `routes/llm/` (6 modules). Split `preprocessingGraph.ts` (1,796 lines) into `preprocessingTools/`. Split `notebookStore.ts` (829 lines) into `stores/notebook/` (7 slices). Split `preprocessingStore.ts` (747 lines) into `stores/preprocessing/` (5 slices). Extract shared modules. Remove ~4,000 lines of dead code. Author 7 architecture documents (1,492 lines).

OpenAI Migration (8+ commits)

Migrate LLM integration to OpenAI GPT-5 Responses API. Model selector UI with catalog support. Streaming tool call delta accumulation and tests. Reasoning effort configuration and model catalog endpoint.

Appendix C: File Change Summary

Area	Changed	Insertions	Deletions	Notes
Frontend	~403	~65,000	~28,000	Incl. lock file (5,344 l.)
Backend	~246	~30,000	~14,000	Incl. lock file (4,099 l.)
Testing	~9	~2,500	~500	NovaCraft fixtures
Docs/Planning	~7	~1,500	~800	7 architecture docs
Config/Other	~8	~200	~500	Build, Docker, CI
Total	673	98,210	43,804	Net: +54,406

Note on raw numbers. Lock files account for approximately 9,443 lines of insertions. NovaCraft fixture CSVs account for approximately 23,781 lines. Excluding these mechanical changes, the human-authored code delta is approximately +65,000 / −43,800—still a substantial net addition reflecting new modules, tests, and documentation.

Report generated March 2026. Data sourced from the merged Sprint 8 branch (MR !27).